

# DATABASE MANAGEMENT SYSTEM

## PROJECT DOCUMENTATION

*[ DATABASE GYM MANAGEMENT PROJECT ]*

|                         |                                                                                                                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Student Name:</b>    | Ahmed Ashraf Mohamed Talba<br>Mohamed Abdel Moneim<br>Abdel Moneim Helal<br>Ziad Ashraf Elsayed Gad<br>Omar Taha Hamed Taha Al-Aqqad<br>Yara Alaa Mohamed Abdel Moneim Othman<br>Shaima Mahmoud Muhammad |
| <b>Student ID:</b>      | 42510385<br>42510908<br>42511196<br>42510626<br>42510788<br>42510826                                                                                                                                     |
| <b>Course:</b>          | Introduction to Database Management Systems                                                                                                                                                              |
| <b>Instructor:</b>      | Dr. Wesam Ahmed                                                                                                                                                                                          |
| <b>Submission Date:</b> | 28/3/2026                                                                                                                                                                                                |

## Table of Contents

### *Catalog*

|                                            |           |
|--------------------------------------------|-----------|
| <b>Table of Contents .....</b>             | <b>2</b>  |
| <b>1. Executive Summary .....</b>          | <b>3</b>  |
| <b>2. Introduction .....</b>               | <b>4</b>  |
| <b>2.1 Project Background .....</b>        | <b>4</b>  |
| <b>2.2 Project Objectives .....</b>        | <b>4</b>  |
| <b>2.3 Scope and Limitations .....</b>     | <b>4</b>  |
| <b>3. Database Design .....</b>            | <b>5</b>  |
| <b>3.1 Conceptual Design .....</b>         | <b>5</b>  |
| <b>3.2 Logical Design .....</b>            | <b>7</b>  |
| <b>3.3 Physical Design .....</b>           | <b>9</b>  |
| <b>4. Implementation .....</b>             | <b>10</b> |
| <b>4.1 Database Creation .....</b>         | <b>10</b> |
| <b>4.2 Sample Data .....</b>               | <b>13</b> |
| <b>4.3 Queries and Operations .....</b>    | <b>14</b> |
| <b>5. Challenges and Solutions .....</b>   | <b>17</b> |
| <b>6. Conclusion and Future Work .....</b> | <b>18</b> |
| <b>6.1 Project Summary .....</b>           | <b>18</b> |
| <b>6.2 Lessons Learned .....</b>           | <b>19</b> |
| <b>6.3 Future Enhancements .....</b>       | <b>19</b> |
| <b>7. References .....</b>                 | <b>20</b> |

## 1. Executive Summary

This project describes the design and implementation of an automated database management system for a gym. It is used to control members, trainers, subscriptions at one place. and workout plans efficiently.

The database, you help save and sort data like member details, membership plans, attendance, and payments.

At the core, it aims to reduce manual effort, increase data accuracy and offer easy access to information.

The system allow to perform action like creating new members, assigning trainers, tracking subscriptions, and generating reports.

We hope that all these together will show you how database systems can be — manage real-world business operations effectively.

## 2. Introduction

### 2.1 Project Background

Gyms need an organized system to manage members, trainers, and subscriptions. Manual systems are inefficient and prone to errors.

### 2.2 Project Objectives

- Store member data efficiently
- Manage subscriptions and payments
- Assign trainers to members
- Track attendance
- Generate reports

### 2.3 Scope and Limitations

#### Scope:

- Managing gym members
- Managing subscriptions and membership plans
- Managing trainers and employees

---

#### Limitations:

- No mobile application is implemented
- The system is not deployed online (offline system only)
- The database uses sample (dummy) data only

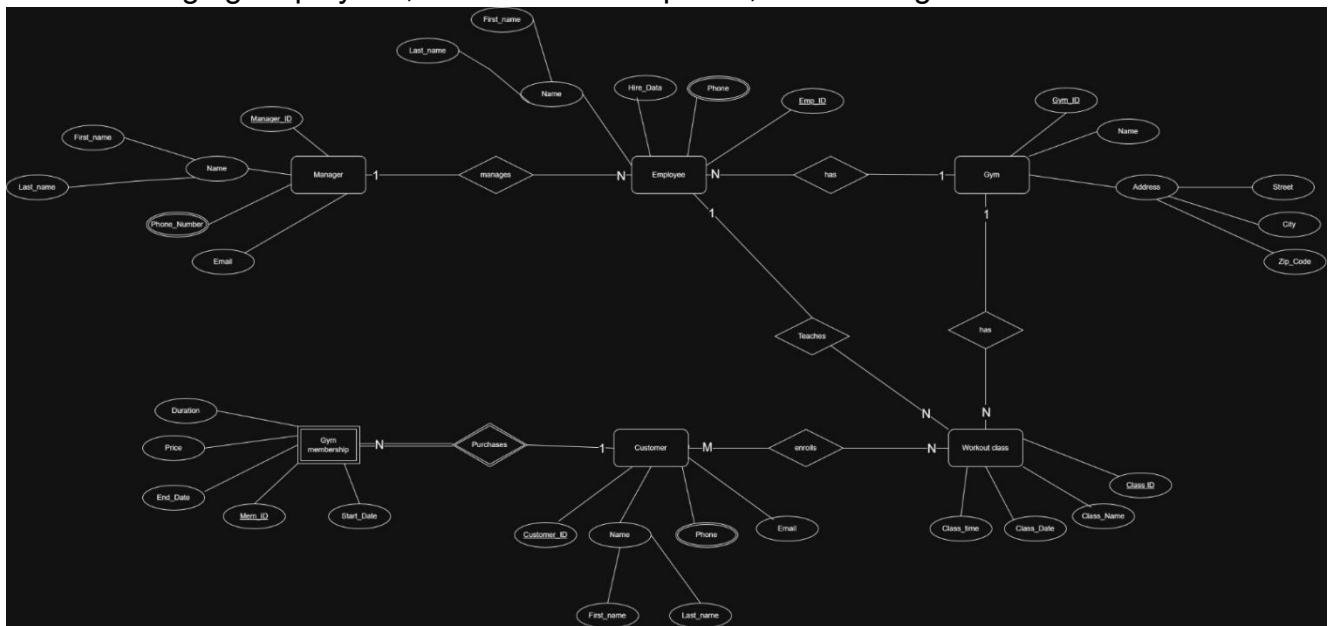
### 3. Database Design

#### 3.1 Conceptual Design

##### 3.1.1 Entity-Relationship (ER) Diagram

The ER diagram illustrates the structure of the gym database system. It includes the main entities: Gym, Manager, Employee, Customer, Membership, and WorkoutClass.

The diagram shows the relationships between these entities such as managing employees, customer subscriptions, and training sessions.



<https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=blank&layers=1&nav=1&title=Untitled%20Diagram.drawio&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1TgpSY0uvWuFkDQwRNqOBaVvmz5eDSYbt4%26export%3Ddownload>

##### 3.1.2 Entity Descriptions

###### 1. Gym

Represents the gym or fitness center.

###### Attributes:

- GymID (Primary Key)
- Name
- Address
  - City
  - Street
  - Zip\_ID

## 2. Manager

Represents the manager responsible for overseeing employees.

### Attributes:

- ManagerID (Primary Key)
  - Name
    - First\_name
    - Last\_name
  - Phone Number
  - Email
- 

## 3. Employee

Represents gym staff such as trainers.

### Attributes:

- EmplID (Primary Key)
  - Name
    - First\_name
    - Last\_name
  - HireDate
  - Phone
- 

## 4. Customer

Represents gym members.

### Attributes:

- CustomerID (Primary Key)
  - Name ( First\_name , Last\_name )
  - Email
  - Phone
- 

## 5. Gym Membership

Represents subscription plans for customers.

### Attributes:

- MemID (Primary Key)
- Start\_Date
- End\_Date
- Price
- Duration

---

## 6. WorkoutClass

Represents fitness classes provided by the gym.

### Attributes:

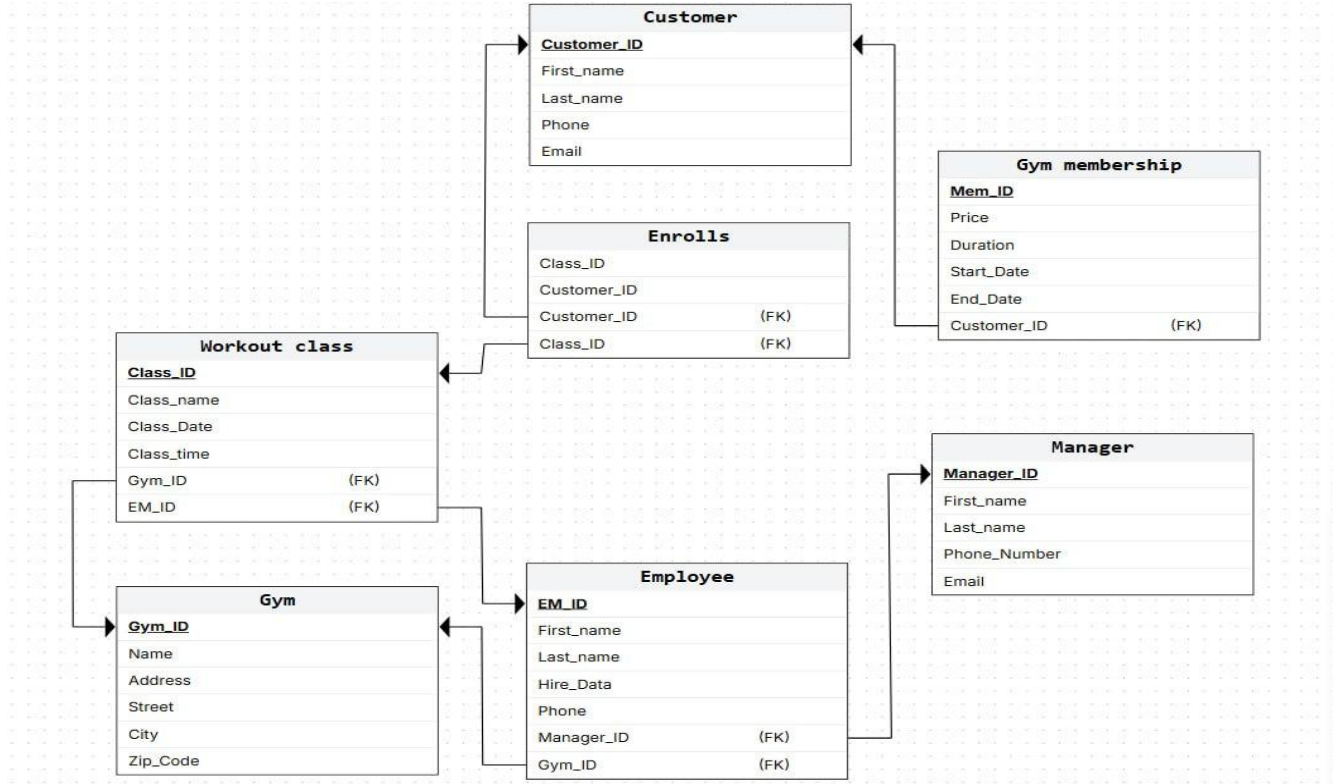
- ClassID (Primary Key)
- ClassName
- ClassDate
- ClassTime

### Relationships

- A **Customer** purchases for a **Membership** (One-to-Many)
- A **Customer** enrolls in **WorkoutClasses** (Many-to-Many)
- A **Gym** has multiple **Memberships** (One-to-Many)
- A **Gym** has multiple **Employees** (One-to-Many)
- A **Manager** manages multiple **Employees** (One-to-Many)
- An **Employee** teaches multiple **WorkoutClasses** (One-to-Many)

## 3.2 Logical Design

### 3.2.1 Relational Schema



### Gym Table

| Attribute Name | Data Type    | Constraints  | Description                    |
|----------------|--------------|--------------|--------------------------------|
| <u>GymID</u>   | INT          | PK, NOT NULL | Unique identifier for each gym |
| Name           | VARCHAR(50)  | NOT NULL     | Name of the gym                |
| Address        | VARCHAR(100) | NOT NULL     | Location of the gym            |

### Manager Table

| Attribute Name   | Data Type    | Constraints  | Description                        |
|------------------|--------------|--------------|------------------------------------|
| <u>ManagerID</u> | INT          | PK, NOT NULL | Unique identifier for each manager |
| Name             | VARCHAR(50)  | NOT NULL     | Manager's name                     |
| Phone Number     | VARCHAR(20)  | NOT NULL     | Manager's phone number             |
| Email            | VARCHAR(100) | NOT NULL     | Manager's Email                    |

### Employee Table

| Attribute Name | Data Type   | Constraints  | Description                          |
|----------------|-------------|--------------|--------------------------------------|
| <u>EmpID</u>   | INT         | PK, NOT NULL | Unique identifier for each employee  |
| Name           | VARCHAR(50) | NOT NULL     | Employee's name                      |
| Phone Number   | VARCHAR(20) | NOT NULL     | Employee's phone number              |
| HireDate       | DATE        | NOT NULL     | The date when the employee was hired |

### Customer Table

| Attribute Name    | Data Type    | Constraints  | Description                         |
|-------------------|--------------|--------------|-------------------------------------|
| <u>CustomerID</u> | INT          | PK, NOT NULL | Unique identifier for each Customer |
| Name              | VARCHAR(50)  | NOT NULL     | Customer's name                     |
| Phone Number      | VARCHAR(20)  | NOT NULL     | Customer's phone number             |
| Email             | VARCHAR(100) | NOT NULL     | Customer's Email                    |

## Membership Table

| Attribute Name | Data Type     | Constraints  | Description                                    |
|----------------|---------------|--------------|------------------------------------------------|
| <u>MemID</u>   | INT           | PK, NOT NULL | Unique identifier for each Membership          |
| Start_Date     | DATE          | NOT NULL     | The date when the membership starts            |
| End_Date       | DATE          | NOT NULL     | The date when the membership Ends              |
| Price          | DECIMAL(10,2) | NOT NULL     | Cost of the membership                         |
| Duration       | INT           | NOT NULL     | Duration of the membership (in days or months) |

## ◆ WorkoutClass Table

| Attribute Name | Data Type   | Constraints  | Description                             |
|----------------|-------------|--------------|-----------------------------------------|
| <u>ClassID</u> | INT         | PK, NOT NULL | Unique identifier for each WorkoutClass |
| ClassName      | VARCHAR(50) | NOT NULL     | Name of the workout class               |
| ClassDate      | DATE        | NOT NULL     | The date when the class takes place     |
| ClassTime      | TIME        | NOT NULL     | The time when the class starts          |

## 3.3 Physical Design

### 3.3.1 Database Management System Selection

MySQL was selected as the database management system due to its simplicity, performance, and wide community support. It is suitable for handling structured data such as gym records and supports all required SQL operations.

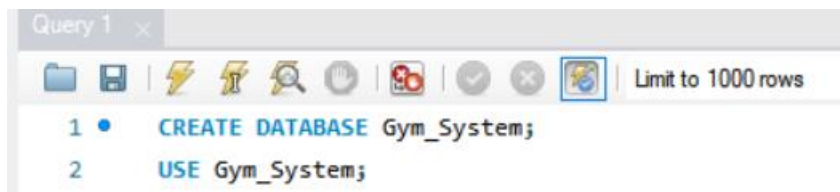
## 4. Implementation

### 4.1 Database Creation

The database was created using MySQL to manage gym operations efficiently. Several tables were designed to store information about customers, managers, employees, gym memberships, workout classes, and enrollments.

The following SQL statements were used to create the database and tables.

#### Create Database



```
Query 1
1 • CREATE DATABASE Gym_System;
2   USE Gym_System;
```

#### Customer Table

```
4   -- Customer Table
5 • CREATE TABLE Customer (
6       Customer_ID INT PRIMARY KEY,
7       First_name VARCHAR(50),
8       Last_name VARCHAR(50),
9       Phone VARCHAR(20),
10      Email VARCHAR(100)
11 );
```

#### Manager Table

```
18   -- Manager Table
19   CREATE TABLE Manager (
20       Manager_ID INT PRIMARY KEY,
21       First_name VARCHAR(50),
22       Last_name VARCHAR(50),
23       Phone_Number VARCHAR(20),
24       Email VARCHAR(100)
25   );
```

## Gym Table

```
31      -- Gym Table
32 • ○ CREATE TABLE Gym (
33     Gym_ID INT PRIMARY KEY,
34     Name VARCHAR(100),
35     Address VARCHAR(100),
36     Street VARCHAR(100),
37     City VARCHAR(50),
38     Zip_Code VARCHAR(10)
39 );
```

## Employee Table

```
45      -- Employee Table
46 • ○ CREATE TABLE Employee (
47     EM_ID INT PRIMARY KEY,
48     First_name VARCHAR(50),
49     Last_name VARCHAR(50),
50     Hire_Date DATE,
51     Phone VARCHAR(20),
52     Manager_ID INT,
53     Gym_ID INT,
54     FOREIGN KEY (Manager_ID) REFERENCES Manager(Manager_ID),
55     FOREIGN KEY (Gym_ID) REFERENCES Gym(Gym_ID)
56 );
57
```

## Gym Membership Table

```
62      -- Gym Membership Table
63 • ○ CREATE TABLE Gym_membership (
64     Mem_ID INT PRIMARY KEY,
65     Price DECIMAL(10,2),
66     Duration VARCHAR(50),
67     Start_Date DATE,
68     End_Date DATE,
69     Customer_ID INT,
70     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
71 );
--
```

## Workout Class Table

```
78      -- Workout Class Table
79  ● ○ CREATE TABLE Workout_class (
80      Class_ID INT PRIMARY KEY,
81      Class_name VARCHAR(100),
82      Class_Date DATE,
83      Class_time TIME,
84      Gym_ID INT,
85      EM_ID INT,
86      FOREIGN KEY (Gym_ID) REFERENCES Gym(Gym_ID),
87      FOREIGN KEY (EM_ID) REFERENCES Employee(EM_ID)
88  );
--
```

## Enrolls Table

```
95      -- Enrolls Table
96  ● ○ CREATE TABLE Enrolls (
97      Customer_ID INT,
98      Class_ID INT,
99      PRIMARY KEY (Customer_ID, Class_ID),
100     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
101     FOREIGN KEY (Class_ID) REFERENCES Workout_class(Class_ID)
102  );
103
```

## 4.2 Sample Data

Sample data was inserted into all tables to test the database functionality and relationships between entities.

The inserted data includes:

- Customers information
- Managers information
- Employees data
- Gym memberships
- Workout classes
- Customer enrollments

```
84
85 • INSERT INTO Customer VALUES
86 (1,'Ahmed','Ashraf','01011111111','ahmed@gmail.com'),
87 (2,'Mahmoud','Hassan','01022222222','mahmoud@gmail.com'),
88 (3,'Omar','Khaled','01033333333','omar@gmail.com');
89
90 • INSERT INTO Manager VALUES
91 (1,'Ziad','Ashraf','01111111111','ziad@gmail.com'),
92 (2,'Yousef','Adel','01122222222','yousef@gmail.com');
93
94 • INSERT INTO Gym VALUES
95 (1,'Giant Fitness Gym','Mobark City','Street 1','Mansoura','12345'),
96 (2,'Mr.X Gym','Mobark City','Street 2','Mansoura','54321');
97
98 • INSERT INTO Employee VALUES
99 (1,'Hatem','Nabil','2023-01-10','01211111111',1,1),
100 (2,'Nour','Muhammad','2023-02-15','01222222222',2,2);
101
102 • INSERT INTO Gym_membership VALUES
103 (1,500,'1 Month','2026-04-01','2026-05-01',1),
104 (2,1200,'3 Months','2026-04-01','2026-07-01',2),
105 (3,2000,'6 Months','2026-04-01','2026-10-01',3);
106
107 • INSERT INTO Workout_class VALUES
108 (1,'Boxing','2026-04-25','10:00:00',1,1),
109 (2,'Cardio','2026-04-26','12:00:00',2,2),
110 (3,'MMA','2026-04-27','05:00:00',1,2);
111
112 • INSERT INTO Enrolls VALUES
113 (1,1),
114 (2,2),
115 (3,3);
```

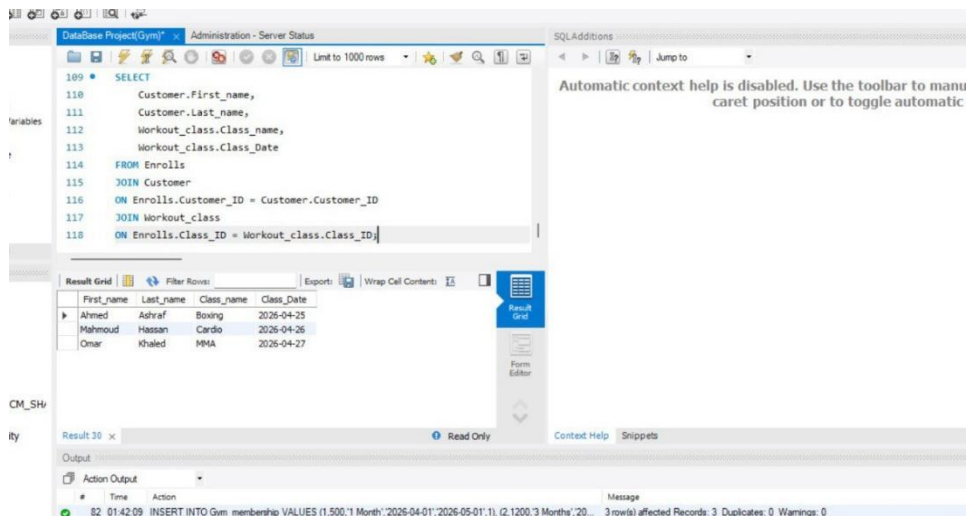
## 4.3 Queries and Operations

### 4.3.1 Complex Queries

Several SQL queries were implemented to test the database functionality and relationships between tables.

#### Query 1:

Purpose: Display customers and the classes they enrolled in



Automatic context help is disabled. Use the toolbar to manually toggle automatic context help.

```
109 SELECT
110     Customer.First_name,
111     Customer.Last_name,
112     Workout_class.Class_name,
113     Workout_class.Class_Date
114 FROM Enrolls
115 JOIN Customer
116 ON Enrolls.Customer_ID = Customer.Customer_ID
117 JOIN Workout_class
118 ON Enrolls.Class_ID = Workout_class.Class_ID;
```

| First_name | Last_name | Class_name | Class_Date |
|------------|-----------|------------|------------|
| Ahmed      | Ashraf    | Boxing     | 2026-04-25 |
| Mahmoud    | Hassan    | Cardio     | 2026-04-26 |
| Omar       | Khaled    | MMA        | 2026-04-27 |

Result 30 x

Read Only

Context Help

Snippets

Output

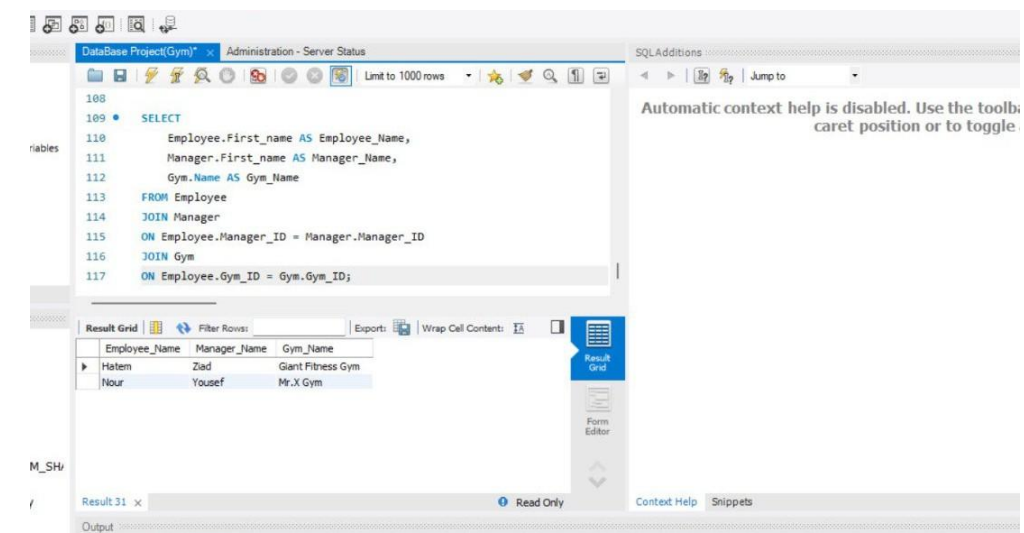
Action Output

Message

82 01:42:09 INSERT INTO Gym\_membership VALUES (1,500,'1 Month','2026-04-01','2026-05-01',1), (2,1200,'3 Months','2026-05-01','2026-07-01',1), (3,1800,'6 Months','2026-07-01','2026-09-01',1); 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0

#### Query 2:

Purpose: Display employees with their manager names and gym names



Automatic context help is disabled. Use the toolbar to manually toggle automatic context help.

```
108
109 SELECT
110     Employee.First_name AS Employee_Name,
111     Manager.First_name AS Manager_Name,
112     Gym.Name AS Gym_Name
113 FROM Employee
114 JOIN Manager
115 ON Employee.Manager_ID = Manager.Manager_ID
116 JOIN Gym
117 ON Employee.Gym_ID = Gym.Gym_ID;
```

| Employee_Name | Manager_Name | Gym_Name          |
|---------------|--------------|-------------------|
| Hatem         | Ziad         | Giant Fitness Gym |
| Nour          | Yousef       | Mr.X Gym          |

Result 31 x

Read Only

Context Help

Snippets

Output

### Query 3:

Purpose: Count number of employees in each gym using GROUP BY

SQL Server Enterprise Manager interface showing Query 3. The query is as follows:

```
106 (2,2),
107 (3,3);
108
109 SELECT
110     Gym.Name,
111     COUNT(Employee.EM_ID) AS Number_Of_Employees
112 FROM Gym
113 JOIN Employee
114 ON Gym.Gym_ID = Employee.Gym_ID
115 GROUP BY Gym.Name;
```

The Result Grid shows the following data:

| Name              | Number_Of_Employees |
|-------------------|---------------------|
| Giant Fitness Gym | 1                   |
| Mr.X Gym          | 1                   |

The Output pane shows the following message:

```
84 01:43:01 INSERT INTO Workout_class VALUES (1,'Boxing','2026-04-25','10:00:00',1,1), (2,'Cardio','2026-04-25','12:00:00',1,1), (3,'Yoga','2026-04-25','14:00:00',1,1); 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
```

### Query 4:

Purpose: Find customers who have memberships costing more than average price

SQL Server Enterprise Manager interface showing Query 4. The query is as follows:

```
109 SELECT
110     First_name,
111     Last_name
112 FROM Customer
113 WHERE Customer_ID IN (
114     SELECT Customer_ID
115     FROM Gym_membership
116     WHERE Price > (
117         SELECT AVG(Price)
118         FROM Gym_membership
119     )
120 )
```

The Result Grid shows the following data:

| First_name | Last_name |
|------------|-----------|
| Omar       | Khaled    |

The Output pane shows the following message:

```
Customer 33 x Read Only Context Help Snippets
Output
Action Output
# Time Action Message
```

## Query 5:

Purpose: Show total membership revenue grouped by duration

The screenshot displays a database management interface with a SQL editor and a results pane. The SQL editor contains the following query:

```
104 • INSERT INTO Enrolls VALUES
105 (1,1),
106 (2,2),
107 (3,3);
108
109 • SELECT
110     Duration,
111     SUM(Price) AS Total_Revenue
112 FROM Gym_membership
113 GROUP BY Duration;
```

The results pane shows a table with two columns: Duration and Total\_Revenue. The data is as follows:

| Duration | Total_Revenue |
|----------|---------------|
| 1 Month  | 500.00        |
| 3 Months | 1200.00       |
| 6 Months | 2000.00       |

The bottom of the interface shows the Action Output pane with the following message:

```
86 01:43:23 INSERT INTO Enrolls VALUES (1,1), (2,2), (3,3) 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
```

## 5. Challenges and Solutions

During the implementation of the Gym Management System database, several challenges were encountered. Different solutions were applied to ensure the database worked correctly and efficiently.

### Challenge 1:

#### Description:

One of the main challenges was connecting the tables correctly using primary keys and foreign keys.

#### Solution:

Foreign key constraints were used to link related tables such as:

- Employee with Manager and Gym
- Gym\_membership with Customer
- Workout\_class with Employee and Gym
- Enrolls with Customer and Workout\_class

This helped maintain data integrity and prevent invalid data entry.

### Challenge 2:

#### Description:

Designing the database structure and deciding how to separate the data into tables was difficult at first.

#### Solution:

The database was normalized by dividing the data into multiple related tables to reduce redundancy and improve organization.

### **Challenge 3:**

#### **Description:**

Some SQL queries required joins, aggregate functions, and subqueries which were challenging during implementation.

#### **Solution:**

Different SQL operations such as:

- JOIN
- GROUP BY
- AVG()
- COUNT()
- Subqueries

were used and tested successfully to retrieve accurate information from the database.

## **6. Conclusion and Future Work**

### **6.1 Project Summary**

The Gym Management System database was successfully designed and implemented using MySQL.

The project manages:

- Customers
- Employees
- Managers
- Gym memberships
- Workout classes
- Customer enrollments

The database supports storing, organizing, and retrieving gym information efficiently through SQL queries and relational tables.

## 6.2 Lessons Learned

Through this project, several important database concepts were learned, including:

- Database design
- Entity relationships
- Primary and foreign keys
- SQL queries
- Data normalization
- Query optimization

The project also improved practical skills in using MySQL and managing relational databases.

## 6.3 Future Enhancements

Several improvements can be added in the future to enhance the system functionality.

### Enhancement 1:

Adding a payment management system for membership fees.

### Enhancement 2:

Creating a graphical user interface (GUI) for easier interaction with the database.

### Enhancement 3:

Adding attendance tracking for workout classes and gym visits.

### Enhancement 4:

Generating reports and statistics for customers and memberships.

## 7. References

1. MySQL Documentation
2. Database Management Systems Course Materials
3. W3Schools SQL Tutorial
4. Back4app Tutorials
5. Oracle MySQL Official Website
6. Lecture Notes and Practical Sessions